Artintele.ai
Networking, Deployment & Hosting

A from-scratch, textbook explanation of how the website runs end to end — the network path, the server, the containers, HTTPS, the login system, and the reasoning behind every decision.

Live system: **https://artintele.ai**

Stack: FastAPI + SQLite · Docker · Caddy · AWS Lightsail (Mumbai) · Namecheap DNS

Prepared as a personal reference.

1. The Big Picture — what happens when someone visits the site

Before the details, here is the entire journey of a single visit to **https://artintele.ai**. Every chapter after this zooms into one of these boxes and explains what it is and why we set it up that way.

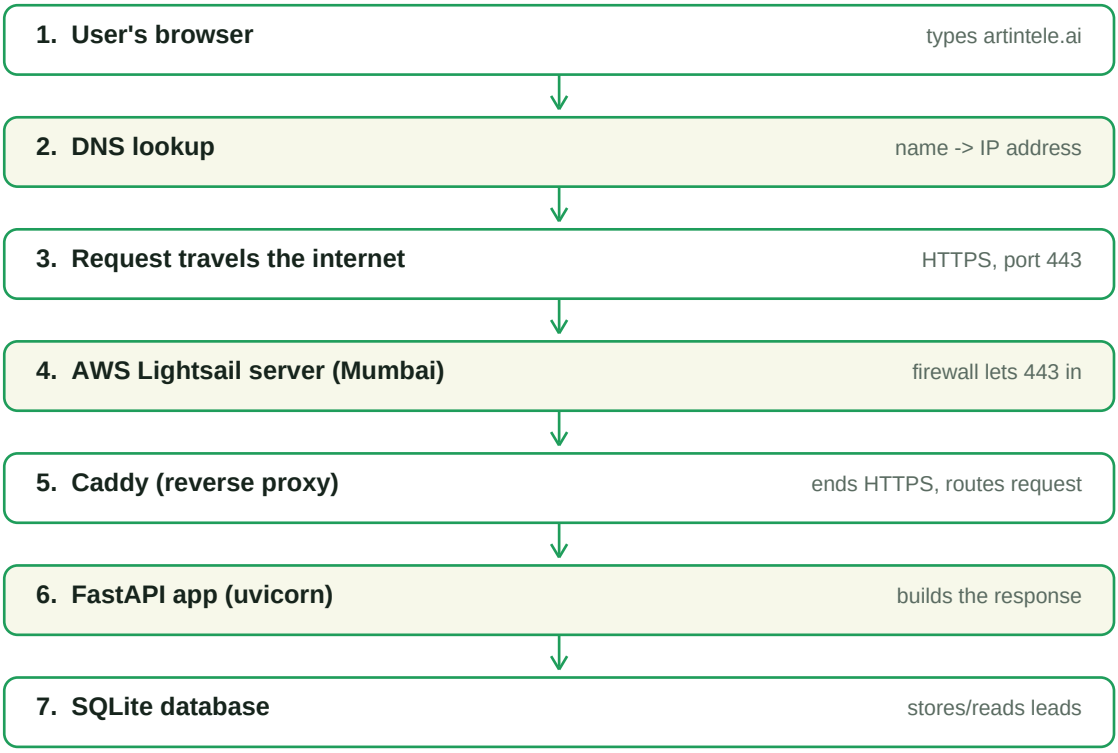

Figure 1 — the request path. The response travels back up the same chain.

In one sentence: a name (the domain) is translated to a number (an IP address), the browser opens a secure connection to that number, a firewall lets it through to our server, a reverse proxy handles the encryption and forwards the request to our application, and the application reads or writes the database and returns the page.

Note — Two mental models worth holding: (a) **a website is just a program running on a computer that is always on and reachable from the internet**; (b) **most of 'hosting' is about addressing (how people find that computer) and trust (proving it's really us, over an encrypted channel).**

2. Networking fundamentals (the concepts in play)

2.1 IP address — the real location

Every computer reachable on the internet has an **IP address**, a number like **52.66.147.115**. That is the actual destination data is sent to. Our server (on AWS) has one. The problem: humans can't remember numbers, and the number can change. That's why domains and DNS exist.

2.2 Domain name — the human-friendly label

artintele.ai is a **domain** — a rentable, memorable name that *points to* an IP address. We bought it from a **registrar** (Namecheap). Owning the domain lets us decide which IP it points to.

2.3 DNS — the internet's phone book

DNS (Domain Name System) is the global lookup service that translates a domain into its current IP address. When someone types artintele.ai, their browser asks DNS "what's the IP for this?" and DNS answers with our server's IP. We control that answer through DNS **records** at Namecheap. The key record types we used:

Record	What it does	Why we used it
A	Maps a name to an IPv4 address	@ **-> server IP** and **www -> server IP** so the domain reaches our box
TXT	Stores text (verification, email auth)	SPF / DKIM / DMARC / Google verification — these run your Workspace **email**
MX	Mail server for the domain	Routes @artintele.ai email to Google — left untouched

Note — This is exactly why we were careful at Namecheap: the TXT and MX records are your **email** configuration. Deleting them would have broken email for @artintele.ai. We only **added** the two A records and left every email record alone.

2.4 Ports — many doors on one address

One server (one IP) runs many services, so each service listens on a numbered **port**. The ones that matter here:

- **Port 80 (HTTP)** — plain, unencrypted web traffic. We allow it, but only so Caddy can redirect it to HTTPS and complete certificate checks.
- **Port 443 (HTTPS)** — encrypted web traffic. This is where the real site is served.
- **Port 22 (SSH)** — the secure remote terminal we use to administer the server.
- **Port 8080** — internal only. Our application listens here, *inside* the server; the public never touches it directly.

2.5 Why a firewall

A **firewall** decides which ports the outside world may reach. On our server we open only **22, 80, and 443** and block everything else. So even though the app listens on 8080 inside, nobody can hit it from outside — they must go through Caddy on 443. Fewer open doors = smaller attack surface.

3. The domain & DNS configuration (what we set, and why)

At Namecheap, under the domain's **Advanced DNS**, we pointed the name at the server:

Type	Host	Value	Purpose
A	@	server IP	the bare domain artintele.ai
A	www	server IP	the www. version

Why both @ and www: to a computer, **artintele.ai** and **www.artintele.ai** are two different hostnames. If only the bare one is configured, visitors typing "www" get nothing. We point both at the server, then (at the proxy layer, Chapter 6) redirect www to the bare domain so there is **one canonical address** — cleaner for users and for SEO.

Why nameservers must be 'Namecheap BasicDNS': the Advanced DNS records only take effect if the domain is actually using Namecheap's DNS servers. If they were switched to a custom provider, our records would be ignored.

Note — DNS changes aren't instant. They **propagate** (spread across the world's DNS caches) over minutes to hours. We verified ours with `dig +short artintele.ai`, which should print the server IP once propagation reaches us.

4. The server — AWS Lightsail

A website needs a computer that is always on and reachable. We rent one: an AWS **Lightsail** instance — Amazon's simplified virtual private server (VPS). A VPS is a slice of a big physical machine that behaves like your own dedicated Linux computer.

4.1 Why a VPS (and not a one-click platform)

We compared Fly.io and Railway (platform-as-a-service) with a VPS. The deciding factors:

- **Payment** — the deciding one. There was no credit card available, and AWS India accepts **UPI** for sign-up and recurring billing. A VPS on AWS was payable.
- **Always-on** — a VPS runs continuously, so there is no 'cold start' delay on the first visit (platforms that sleep to save money add a wake-up lag).
- **Control** — full Linux box: we can run Docker, our own reverse proxy, anything.

4.2 The specific choices and their reasons

Choice	Why
Region: Mumbai (ap-south-1)	Lowest latency for the India audience — data travels less distance
OS: Ubuntu 22.04 LTS	Stable, long-term-support Linux; the standard for servers
Plan: the $5 tier (512 MB RAM)	Cheapest that runs this small app; sized to the workload
Static IP attached	A normal IP can change on restart and break DNS; a static IP is permanent

Choice	Why
2 GB swap file added	512 MB RAM is too little to *build* the app image (pandas). Swap = disk used as overflow memory, preventing the build from being killed
Firewall: 22, 80, 443 only	Minimum doors open; everything else blocked

Note — Cost: AWS's new free tier gives ~$100 of credits over ~5 months, so the first months are effectively free; after that the plan is about $5/month. The reminder for later: switch on UPI AutoPay before the credits/period end so the site doesn't pause.

5. Containerization — Docker

Rather than install Python, libraries, and configuration directly on the server (fragile, hard to reproduce), we package the app in **Docker containers**. A container bundles the app with everything it needs to run, so it behaves identically on any machine. Updates and rollbacks become a single command.

We run **two** containers, defined together in `docker-compose.yml`:

- **app** — the FastAPI application (served by uvicorn) listening on the internal port 8080.
- **caddy** — the reverse proxy / web server facing the internet on ports 80 and 443.

Crucially, the database and uploaded files are stored in Docker **volumes** (`app_data`, `app_uploads`). Volumes live *outside* the container's lifecycle, so rebuilding or updating the app **never loses the leads**. This is why a redeploy is safe.

Note — Why this matters: 'it works on my machine' problems disappear. The exact same image that was built runs on the server, and the leads database survives every redeploy because it lives in a volume, not inside the container.

6. The reverse proxy & HTTPS — Caddy

Caddy sits in front of the application. A **reverse proxy** is a receptionist: it accepts all incoming web requests, handles encryption, and forwards them to the right internal service. We chose Caddy because it provides **automatic HTTPS** with almost no configuration.

6.1 Why HTTPS is non-negotiable here

- Browsers mark plain HTTP sites 'Not secure' and warn users — bad for a business.
- **Google login (OAuth) refuses to work over HTTP** — it requires an https:// address. So without HTTPS, the CRM login could not exist.
- Encryption protects form submissions (names, phone numbers, emails) in transit.

6.2 How the certificate works (and why a domain was required first)

HTTPS needs a **TLS certificate** — a cryptographic proof, issued by a trusted authority (**Let's Encrypt**), that we control this domain. Caddy obtains and auto-renews it. The catch: a certificate can only be issued for a **domain name**, never for a bare IP address. To prove control, Let's Encrypt checks the domain — which only works once DNS points the domain at our server. **That is why the order was: server -> point the domain -> HTTPS issues automatically.**

6.3 What our Caddy config does

```
artintele.ai {
    encode gzip                # compress responses
    header { ... security headers ... }
    @static path /assets/*
    header @static Cache-Control "...immutable"   # cache static files
    reverse_proxy app:8080      # forward to the FastAPI app
}

www.artintele.ai {
    redir https://artintele.ai{uri} permanent     # www -> apex
}
```

So Caddy: terminates HTTPS, compresses, sets security + cache headers, forwards real requests to the app on 8080, and 301-redirects the www hostname to the canonical bare domain.

7. The application layer — FastAPI + SQLite

The **app** container runs a FastAPI program (Python). It does three jobs:

- **Serves the pages** — home, pricing, contact, careers (static HTML/CSS/JS).
- **Runs the lead API** — the contact form POSTs to it; it validates and stores the lead.
- **Serves SEO routes** — it generates `/robots.txt`, `/sitemap.xml`, and `/llms.txt` dynamically.

Data lives in **SQLite** — a lightweight, file-based database kept on the persistent volume. For the current scale (a lead form and a CRM list) it is more than enough; it only struggles under hundreds of simultaneous writes, which a lead form never approaches. If volume ever demands it, the path is to move to PostgreSQL — a 'good problem' for much later.

Why this serves 100+ users easily: the pages are mostly static and cheap to deliver, and a lead submission is one quick database write. The small server handles this with large headroom.

8. Authentication & the CRM

The dashboard at `/dashboard` holds prospect data, so it is locked behind **Google sign-in (OAuth 2.0)**, restricted to **@prachalas.com** accounts. The flow:

- 1. A user visits /dashboard. With no session, the app redirects them to /login.
- 2. They click 'Sign in with Google' and authenticate with Google directly (we never see their password).
- 3. Google redirects back to our **redirect URI**, https://artintele.ai/auth/callback, with a one-time code.
- 4. The app exchanges that code (using the Client ID + Secret) for the user's verified email.
- 5. The app checks the email ends in **@prachalas.com**; if so, it creates a session; if not, access is denied.

Why OAuth instead of our own passwords: we never store or handle passwords (a security liability); Google does the hard part; and we pigg-back on the company's existing Workspace accounts. **Why the domain lock:** the CRM is internal — only company accounts should ever see lead data. The check is done server-side on Google's *verified* email, so it can't be faked.

Note — The Client ID is public by nature; the **Client Secret** is sensitive and lives only in the server's `.env` file (never in the code or Git). The redirect URI is locked to our exact domain, so the credentials can't be used from anywhere else.

9. Security hardening — the HTTP headers

On top of HTTPS, Caddy adds response **headers** that instruct the browser to behave more safely. Each one closes a specific class of attack:

Header	What it does / why
Strict-Transport-Security (HSTS)	Forces browsers to always use HTTPS for the domain — blocks downgrade attacks
X-Frame-Options: SAMEORIGIN	Stops other sites embedding ours in an invisible frame (clickjacking)
X-Content-Type-Options: nosniff	Stops the browser guessing file types — blocks a class of injection tricks
Referrer-Policy	Limits how much of our URL leaks to other sites when a user clicks away
Cross-Origin-Opener-Policy	Isolates our page from other browser tabs/windows

Why we deliberately skipped CSP (Content-Security-Policy) for now: a strict CSP is powerful but would block our inline styles, the inline theme script, Google Fonts, and the canvas animation — breaking the site. It needs careful testing in 'report-only' mode first. We judged it not worth risking a working site for one optional, unscored flag.

10. Performance & caching

- **Cache-Control on /assets/*** — tells browsers to keep CSS/JS/images for a long time, so repeat visits load instantly. Safe because asset URLs carry a version tag (?v=…); changing a file changes its URL and busts the cache.
- **Gzip compression** — Caddy compresses text responses, so less data travels.
- **Optimized assets** — e.g. the logo was reduced from 47.6 KB to 17 KB for a faster first paint.
- **Always-on server** — no cold-start lag (a trade-off we chose over the cheaper 'sleep when idle' mode).

Result: the site scores 99/100 on performance with all Core Web Vitals green.

11. The deployment procedure (and what each step did)

The exact sequence we ran, end to end:

Step	What & why
1. Create the Lightsail instance	The always-on Linux computer (Ubuntu, Mumbai, $5)
2. Attach a static IP	A permanent address so DNS stays valid across restarts
3. Open firewall ports 80/443	Let web traffic reach the server (22 was already open)
4. Point DNS (A @ and www)	Make artintele.ai resolve to the server's IP
5. SSH into the server	Open a remote terminal to administer it
6. Add 2 GB swap	Prevent the build from running out of memory
7. Clone the repo (GitHub token)	Bring the code onto the server
8. Install Docker	The engine that runs the containers

Step	What & why
9. Write the .env file	Server-only secrets: session key, OAuth creds, domain
10. docker compose up --build	Build + start the app and Caddy; HTTPS auto-issues
11. Create Google OAuth client	Enable the dashboard login (redirect to /auth/callback)
12. Add www redirect + headers	Canonical URL + security/cache hardening, then reload Caddy

Updating later is just: push code to GitHub, then on the server `git pull` and `docker compose up -d --build`. The volumes keep all data through the update.

Note — A real snag we hit: a **git pull was blocked** because we had hand-edited a config file on the server, which conflicted with the incoming version. The fix was to discard the local edit (`git checkout -- <file>`) and pull again. Lesson: make changes in the repo and deploy them, rather than editing files directly on the server.

12. Quick glossary

Term	Plain-English meaning
DNS	The system that turns a domain name into an IP address
A record	A DNS entry mapping a name to an IPv4 address
IP address	The numeric address of a computer on the internet
Port	A numbered 'door' on a server; each service uses its own
Firewall	Rules controlling which ports the outside world can reach
VPS	Virtual private server — a rented, always-on Linux computer
Docker / container	A package of an app + its dependencies that runs identically anywhere
Volume	Storage that lives outside a container so data survives redeploys
Reverse proxy	A front-door server that handles HTTPS and routes to internal apps
TLS certificate	Cryptographic proof of domain ownership that enables HTTPS
Let's Encrypt	A free authority that issues TLS certificates automatically
HSTS	A header forcing browsers to always use HTTPS
OAuth	Logging in via a provider (Google) without handling passwords yourself
Redirect URI	The exact URL a provider sends users back to after login
.env file	A server-only file holding secrets, never committed to Git

End of document.